

Quantum Physics by Simulation

A walk through my code for the BPhO Computational Challenge

Bryan

August 14, 2026

Abstract

This is a write-up of the code I wrote for the ten tasks of the BPhO Computational Challenge. For each task I explain what I had to build, the shape I gave the program, and why the interesting lines are written the way they are. Along the way I have tried to be honest about the parts that gave me trouble: the collision detection in the Brownian motion simulation, the unit conversion in the Planck spectrum, and the problem of choosing sensible axes for orbitals that vary in size by a factor of twenty. Every task was also given interactive controls, and all ten were then folded into one application hosted on Streamlit Community Cloud.

Contents

1	How the code is put together	2
2	Task 1: Random walk	2
3	Task 2: Brownian motion	3
4	Task 3: Planck black body spectrum and heat capacity	6
5	Task 4: Photoelectric effect	7
6	Task 5: Hydrogen emission spectrum and Bohr's model	8
7	Task 6: Electron diffraction	10
8	Task 7: Particle in a box	11
9	Task 8: Quantum cryptography	12
10	Task 9: Compton scattering	13
11	Task 10: Hydrogenic orbitals, two dimensional slices	14
12	Task 10: Hydrogenic orbitals, three dimensional	18
13	Extension: making all ten interactive	19

1 How the code is put together

All ten programs are written in Python with NumPy for the arithmetic and Matplotlib for the figures. Beyond that I kept to two habits that made the later tasks much easier than the earlier ones.

The first is that the physics always lives in a plain function that takes numbers and returns numbers. `planckIrradiance(wavelength, T)`, `ringRadius(V, d)` and `classicalMismatch(theta, phi)` know nothing about figures, axes or colours. Everything to do with drawing happens outside them. That separation was not something I planned from the start, but by about Task 4 it was obvious that it made the code far easier to check, because I could call the function on its own and compare the answer against something worked out by hand. It also turned out to be the reason the interactive versions took so little work later on.

The second is that units are handled once, at a single obvious place, and never again. Constants go in as SI, the maths is done in SI, and the conversion to whatever unit makes a readable axis happens right at the boundary. Task 3 is where this nearly caught me out, and I have described that properly in its own section.

I have also tried to name things after the quantity rather than the symbol. `stoppingVoltage` takes more keystrokes than `V`, and `smallSpeedAlongNormal` is a mouthful, but when I came back to the collision code after a fortnight away I could still read it.

2 Task 1: Random walk

The task asks for a walk of N steps of fixed size s , each taken at an angle θ drawn uniformly from 0 to 2π . Written out, the position after k steps is

$$x_k = \sum_{i=1}^k s \cos \theta_i, \quad y_k = \sum_{i=1}^k s \sin \theta_i. \quad (1)$$

My first instinct was to write it as a loop: start at the origin, pick an angle, add $s \cos \theta$ to x , add $s \sin \theta$ to y , store the result, go again. That works, but it is a thousand trips around the loop per walk and I wanted to overlay fifty walks, so fifty thousand trips in total, all to do fifty thousand additions.

What made me rewrite it is noticing that the angles do not depend on where the walker currently is. Nothing about step 700 needs to know the outcome of step 699. So all N angles can be drawn up front, and the summation above is just a running total, which is exactly what `np.cumsum` produces. The whole walk then comes out without looping over steps at all:

```
for i in range(numWalks):
    angles = np.random.uniform(0, 2*np.pi, N)
    x = np.concatenate([[0], np.cumsum(s * np.cos(angles))])
    y = np.concatenate([[0], np.cumsum(s * np.sin(angles))])
    ax.plot(x, y, linewidth=0.5, alpha=0.6)
```

Reading it from the inside out, `np.random.uniform(0, 2*np.pi, N)` hands back the whole array of angles in one call. `s * np.cos(angles)` is then every horizontal displacement at once, and `np.cumsum` turns those displacements into positions. The loop that remains runs fifty times, once per walk, because each walk needs its own line on the axes.

The `np.concatenate([[0], ...])` is small but I would have got it wrong without thinking about it. A cumulative sum over N displacements gives N values, and the first of those is the position after the first step has already been taken. The walk actually has $N + 1$ points because it starts at the origin, so the origin has to be stuck on the front by hand. Leave it out and every path begins one step away from the marker that claims to show the start.

3 Task 2: Brownian motion

This was much the longest task to write. It asks for N small particles of mass m and radius r moving randomly, some of which collide with a large particle of mass M and radius R , with the small particles following a random walk and the collisions handled by conservation of momentum. Three separate problems had to be sorted out before anything sensible appeared on screen.

Not checking every pair

The honest version of this simulation checks every pair of particles for a collision on every frame. With a thousand small particles that is around half a million checks per frame and two thousand frames to get through. I tried it and it was unusable. Almost all of that effort also goes on collisions between small particles, which are not what the task is actually about.

So I modelled those statistically instead. Each small particle flies in a straight line for a set number of steps, which stands in for one mean free path, and then has its direction randomised. That gives each small particle a random walk with a sensible mean free path without ever computing a single small particle collision. `Kn` sets how many steps that is, and the timestep is defined in terms of it:

```
speed = np.sqrt(3 * kB * T / smallMass)
speed = speed / 1000    # nm/ps
Kn = 15
dt = 0.1 * Kn * smallRadius / speed    # timestep in ps
```

The speed comes from equipartition, $\frac{3}{2}k_B T = \frac{1}{2}mv^2$, so the gas is at the right temperature. Tying `dt` to the mean free path rather than picking a round number means that if I change the temperature or the particle mass, the speed changes and the timestep quietly follows, and the simulation still resolves the motion with about the same number of steps. I only realised this was worth doing after changing the temperature once and watching everything fall apart.

The direction randomisation is done to the whole array at once rather than particle by particle:

```
stepsSinceChange += 1
needsNewDirection = stepsSinceChange >= Kn
numNeedingChange = np.sum(needsNewDirection)
if numNeedingChange > 0:
    newAngles = np.random.uniform(0, 2*np.pi, numNeedingChange)
    smallVX[needsNewDirection] = speed * np.cos(newAngles)
    smallVY[needsNewDirection] = speed * np.sin(newAngles)
    stepsSinceChange[needsNewDirection] = 0
```

`needsNewDirection` is a boolean array with one entry per particle, and it gets used twice. `np.sum` on a boolean array counts the `True` values, which tells me how many fresh angles to draw, and then the same array is used as an index to write those angles into exactly the right slots. Only the direction changes and the speed stays put, which is what I want for collisions between identical particles, and it keeps the gas at a constant temperature over the run.

The box edges wrap rather than reflect:

```
smallX = np.where(smallX > boxSize, -boxSize, smallX)
smallX = np.where(smallX < -boxSize, boxSize, smallX)
```

A particle that leaves one side comes back in on the other, so the number of particles in the box never changes and the large particle keeps getting hit at the same rate all the way through.

Reflecting walls would have worked too, but they slowly build up a bias towards the centre of the box over a long run, which is an artefact of the walls rather than anything real.

Stopping particles passing straight through

This is the bug that took longest to find. Collisions are spotted by asking whether the separation of the centres is less than the sum of the radii, but particles move in discrete jumps of `speed * dt`, so a particle can end a step already overlapping the large one, or in principle pass across it entirely and be tested only at positions on either side. The step here is about 1.5 small radii against a large particle 16 small radii across, so a clean pass through the middle cannot actually happen at this timestep, but that is a consequence of how `dt` was chosen rather than anything the collision test enforces, and overlaps at the edge happen constantly. The symptom when they are mishandled is not an error message, it is a large particle that just sits there looking far too calm.

The fix is to catch any particle found inside the large one and push it radially back out onto the surface before doing anything else:

```
dx = smallX - bigX
dy = smallY - bigY
distances = np.sqrt(dx**2 + dy**2)
tooClose = distances < (smallRadius + bigRadius)

collidingIndices = np.where(tooClose)[0]

if collidingIndices.size > 0:
    # push them out to the surface
    smallX[tooClose] = bigX + (smallRadius + bigRadius) * dx[tooClose] /
    distances[tooClose]
    smallY[tooClose] = bigY + (smallRadius + bigRadius) * dy[tooClose] /
    distances[tooClose]
```

Dividing `dx` by `distances` gives a unit vector pointing from the big particle to the small one, and multiplying that by the contact distance puts the particle exactly on the surface. It is a correction to the position and nothing else, so the momentum exchange that comes next is unaffected, and because the move is purely radial the contact normal does not change either.

One ordering detail matters more than it looks. `collidingIndices` is taken from the overlap test before the particles are pushed out, and that is the list handed to the collision response. If the indices were worked out again afterwards, every particle would be sitting at exactly the contact distance rather than inside it, the strict `<` would come back false for all of them, and the large particle would never get kicked at all.

The collision itself, done in the zero momentum frame

Only the components along the line joining the two centres change in a collision between smooth spheres. The components at right angles to it have nothing to act on them.

I could have solved conservation of momentum and kinetic energy simultaneously and substituted, but the algebra is unpleasant and easy to mistype. Working in the zero momentum frame is much cleaner, because in that frame the total momentum is zero and an elastic collision simply reverses both velocities. The frame moves along the normal at

$$v_{\text{ZMF}} = \frac{mu_m + Mu_M}{m + M}, \quad (2)$$

and transforming in, reversing and transforming back gives one line per particle:

$$v'_m = C(v_{\text{ZMF}} - u_m) + v_{\text{ZMF}}, \quad v'_M = C(v_{\text{ZMF}} - u_M) + v_{\text{ZMF}}. \quad (3)$$

C is the coefficient of restitution. Setting it to 1 gives a perfectly elastic collision, which is what I want here, but I left it as a variable at the top of the file rather than writing the elastic case directly into the formula, so that the inelastic case is one number away.

Here is how it goes into code, inside the update function, looping over the handful of indices that are actually touching:

```
for i in collidingIndices:
    normalX = (bigX - smallX[i]) / distances[i]
    normalY = (bigY - smallY[i]) / distances[i]

    smallSpeedAlongNormal = smallVX[i]*normalX + smallVY[i]*normalY
    bigSpeedAlongNormal = bigVX*normalX + bigVY*normalY

    zmfVelocity = (smallMass*smallSpeedAlongNormal + bigMass*bigSpeedAlongNormal) /
    (smallMass + bigMass)

    newSmallSpeedAlongNormal = C*(zmfVelocity - smallSpeedAlongNormal) + zmfVelocity
    newBigSpeedAlongNormal = C*(zmfVelocity - bigSpeedAlongNormal) + zmfVelocity

    deltaSmall = newSmallSpeedAlongNormal - smallSpeedAlongNormal
    deltaBig = newBigSpeedAlongNormal - bigSpeedAlongNormal

    smallVX[i] += deltaSmall * normalX
    smallVY[i] += deltaSmall * normalY
    bigVX += deltaBig * normalX
    bigVY += deltaBig * normalY
```

The structure is what keeps a two dimensional collision solvable with one dimensional formulae. The two dot products, `smallVX[i]*normalX + smallVY[i]*normalY` and its partner, project each velocity onto the normal. The zero momentum frame algebra then works on those single numbers. Finally the *changes* in the normal components get added back as vectors along the normal.

Adding the change rather than overwriting the velocity is the part I would point at if I had to pick one line. It means the sideways component of the velocity is left completely alone without me having to extract it, store it and put it back, because whatever it was before, nothing has been added to it.

The last few lines of the loop then hand the small particle a brand new random direction, which is the same mean free path idea applied at the moment of a bounce:

```
newAngle = np.random.uniform(0, 2*np.pi)
smallVX[i] = speed * np.cos(newAngle)
smallVY[i] = speed * np.sin(newAngle)
stepsSinceChange[i] = 0
```

The mass ratio does the physical work. With `smallMass = 1e-20` and `bigMass = 1e-18` the frame velocity sits very close to the big particle's own velocity, so each individual hit barely moves it. It only goes anywhere because the hits arrive from random directions and do not quite cancel. The trail that accumulates in `bigXHistory` and `bigYHistory` therefore ends up being a random walk that I never wrote as one, which is a nice link back to Task 1.

A note on the numbers

The masses and radii are not real. A genuine Brownian particle is something like 10^{10} times heavier than the molecules hitting it, and simulating that faithfully gives an animation in which

nothing appears to happen. I used a mass ratio of 100 and a radius ratio of 8, which keeps the behaviour I want to show while making the motion visible in a few hundred frames. I would rather state that plainly than present the numbers as though they were physical.

4 Task 3: Planck black body spectrum and heat capacity

Two panels here. The first plots

$$B(\lambda, T) = \frac{2hc^2}{\lambda^5} \cdot \frac{1}{\exp\left(\frac{hc}{\lambda k_B T}\right) - 1} \quad (4)$$

for several temperatures, and the second plots Einstein's model of molar heat capacity for gold, copper and iron.

The unit conversion that nearly slipped past

The Planck formula returns spectral irradiance per metre of wavelength. I wanted the horizontal axis in nanometres, because the visible range is then a readable 380 to 700 rather than a couple of exponents. But the vertical axis is a density with respect to wavelength, so changing the wavelength unit changes the number by a factor of 10^9 .

What makes this dangerous is that the shape of the curve does not care. Plot SI values against a nanometre axis and you get a graph that looks completely correct with a vertical axis that is wrong by nine orders of magnitude, and nothing about the figure tells you. I only spotted it by checking the peak value against a number I already knew.

The conversion therefore happens inside the function, so that what comes out always matches the axis it is about to be drawn on:

```
def planckIrradiance(wavelength, T):
    # Planck's formula gives spectral irradiance in W/m^2 per metre of wavelength
    # Dividing by 1e9 converts this to W/m^2 per nanometre for a cleaner axis
    exponent = (h * c) / (wavelength * kB * T)
    BperMetre = (2 * h * c**2 / wavelength**5) * (1 / (np.exp(exponent) - 1))
    return BperMetre * 1e-9
```

Wavelength goes in as metres, irradiance per nanometre comes out, and the only other conversion in the task is the `wavelengths * 1e9` in the plotting call. If the axis ever looks wrong there are exactly two places to check.

Using Wien's law as a check rather than an annotation

Instead of eyeballing where each curve peaks, I calculate it from $\lambda_{\max} = b/T$ and mark it:

```
peakWavelength = wienPeakWavelength(T) * 1e9
peakIrradiance = planckIrradiance(wienPeakWavelength(T), T)
ax1.plot(peakWavelength, peakIrradiance, 'o', color=col, markersize=5)
ax1.annotate(f'{peakWavelength:.0f}nm',
             xy=(peakWavelength, peakIrradiance),
             xytext=(peakWavelength + 100, peakIrradiance),
             color=col, fontsize=7)
```

This started out as a label and turned into my main test of the whole function. Wien's law comes from differentiating the Planck function and setting the result to zero, so it is not an

independent fact I am adding to the plot, it is a consequence of the same expression the code is evaluating. If I had mistyped the exponent, the power of λ , or one of the constants, the marker would sit somewhere other than the top of my curve and I would see it immediately. It landing exactly on the summit for all three temperatures is the reason I trust the implementation. Running it at 5778 K, the surface temperature of the Sun, the marked peak and the numerical maximum of the curve agree at 501 nm to within a tenth of a per cent.

The shaded band from `axvspan(380, 700, ...)` is there for context, so the curves can be read against the range the eye responds to rather than in isolation.

The heat capacity panel

The second panel is the Einstein model,

$$C_V = 3R u^2 \frac{e^u}{(e^u - 1)^2}, \quad u = \frac{\theta_E}{T}, \quad (5)$$

with θ_E taken as 165 K for gold, 343 K for copper and 470 K for iron. Strictly these are the Debye temperatures of the three metals, which are the values that are readily tabulated; the Einstein temperature of a given material is somewhat lower, typically around three quarters of it. Using them here shifts each curve slightly along the temperature axis but changes nothing about the shape or the ordering of the three materials, which is what the panel is for.

The materials sit in a dictionary so that adding another one is a single line rather than another block of plotting code:

```
materials = {
    'Gold': 165,
    'Copper': 343,
    'Iron': 470
}

for material, Te in materials.items():
    u = Te / tempRange
    Cv = 3 * R * u**2 * np.exp(u) / (np.exp(u) - 1)**2
    ax2.plot(tempRange, Cv, label=f'{material} (T_E = {Te}K)', linewidth=1.5)

ax2.axhline(y=3*R, color='gray', linestyle='--', linewidth=1,
            label=f'Dulong-Petit limit 3R = {3*R:.1f} J/mol/K')
```

`u = Te / tempRange` divides a single number by an array of a thousand temperatures and gives an array back, so the entire curve is produced by three lines of arithmetic with no loop over temperature.

The dashed reference line is worth the one line it costs. Without something to compare against, three rising curves do not obviously mean anything. With $3R$ drawn across the plot it is immediately clear that all three approach it from below and that the approach happens at a different temperature for each material.

5 Task 4: Photoelectric effect

Stopping voltage against frequency, for a set of metals. Einstein's equation rearranges to

$$V_s = \frac{h}{e} f - \frac{\phi}{e}, \quad (6)$$

which is a straight line whose gradient h/e is the same for every metal and whose intercept depends only on the work function. The nine metals live in a dictionary keyed by name so that

the labels come straight out of the same place as the numbers, with no risk of a legend that disagrees with the curve it is describing.

The only real decision in the task is what to do below the threshold frequency. The equation happily returns negative voltages down there, but the physics does not: below $f_0 = \phi/h$ the photon cannot free an electron and nothing is emitted at all, so there is no stopping voltage to plot. Drawing the line anyway would imply an effect that does not exist.

I deal with it by replacing everything below threshold with `np.nan`, because Matplotlib draws nothing where the data is not a number:

```
for metal, workFunction in metals.items():
    thresholdFreq = (workFunction * e) / h

    # np.nan below threshold so line only appears where emission actually occurs
    stoppingVoltage = np.where(
        freqRange > thresholdFreq,
        (h * freqRange - workFunction * e) / e,
        np.nan
    )
    ax.plot(freqRange * 1e-15, stoppingVoltage, linewidth=1.2, label=...)
```

`np.where` works elementwise across the whole frequency array, checking the condition and picking from one branch or the other for each entry, so no loop is needed. Each line then starts abruptly at its own threshold, which is exactly what the experiment shows and is the feature worth seeing.

The horizontal axis is plotted as `freqRange * 1e-15` so the numbers read as 0 to 2 rather than 0 to 2×10^{15} , with the factor stated in the axis label. Same principle as Task 3, applied somewhere much less dangerous.

6 Task 5: Hydrogen emission spectrum and Bohr's model

The task asks for a graph of photon energy against wavelength for emissions caused by transitions between hydrogen energy levels. A photon carries away the gap between the two levels, and its wavelength follows from that:

$$E_\gamma = E_{n_i} - E_{n_f}, \quad \lambda = \frac{hc}{E_\gamma}. \quad (7)$$

The level energies come from **hydrogenic**, a module I wrote separately and used in both this task and Task 10. Keeping it in one file means the two tasks are guaranteed to be talking about the same atom rather than two slightly different ones that happen to share a name.

```
from hydrogenic import orbitalEnergy
```

What the module is actually computing

`orbitalEnergy` evaluates

$$E_n = -\frac{\mu e^4 Z^2}{8\epsilon_0^2 h^2 n^2}, \quad (8)$$

and the piece worth pointing at is μ rather than m_e . The nucleus is not infinitely heavy, so both particles orbit their shared centre of mass and the correct mass to use is the reduced mass

$$\mu = \frac{m_e M}{m_e + M}, \quad (9)$$

which is what `reducedMass(A)` returns, with $M = Au$. For hydrogen the correction is about one part in two thousand, so it changes nothing I could see on this figure. I put it in because the same function is called with Z and A from the interactive controls, and for a heavier hydrogenic ion the correction stops being negligible. Writing the general case once was cheaper than writing the hydrogen case now and discovering the limitation later.

The module goes considerably further than this task needs. It also builds the full wavefunction, with the radial part

$$R_{n\ell}(r) \propto x^\ell e^{-x/2} L_{n-\ell-1}^{2\ell+1}(x), \quad x = \frac{2r}{an}, \quad (10)$$

where the associated Laguerre polynomial is evaluated by `laguerreSum` as its explicit finite sum rather than through a library call, and the angular part from the spherical harmonics $Y_\ell^m(\theta, \phi)$, assembled in `sphericalHarmonic` from `scipy.special.lpmv` for the associated Legendre function together with the normalisation and the $e^{im\phi}$ factor. `angularPart` then combines Y_ℓ^m and Y_ℓ^{-m} into real-valued pairs so the orbitals come out as the familiar lobes rather than complex-valued objects that cannot be plotted directly, and `probabilityDensity` applies the Born interpretation, $|\psi|^2 = \psi^* \psi$. None of that is exercised here, where only the energies matter, but all of it is what Task 10 draws. I have described those routines properly in the Task 10 section.

The structural decision here is to organise everything by series. Every transition that ends on the same lower level belongs to one named family, so those go in a dictionary that carries both the final level and the colour, and the whole figure is then built by looping over it:

```
series = {
    'Lyman':    {'nFinal': 1, 'colour': 'magenta'},
    'Balmer':   {'nFinal': 2, 'colour': 'red'},
    'Paschen':  {'nFinal': 3, 'colour': 'blue'},
    'Brackett': {'nFinal': 4, 'colour': 'lime'},
    'Pfund':    {'nFinal': 5, 'colour': 'white'},
}
```

Keeping the colour next to the final level rather than in a separate list means the two can never drift out of step, which is a mistake I made in an earlier version where a fourth colour quietly labelled the fifth series.

Each transition is drawn as a vertical line rather than a point, so the figure reads like something off a spectrometer, with dots along the tops to pick out the shape of each family:

```
for nInitial in range(nFinal + 1, nMax + 1):
    # Photon energy is the gap between the two levels
    photonEnergy = orbitalEnergy(Z, A, nInitial) - orbitalEnergy(Z, A, nFinal)
    wavelength = (h * c) / (photonEnergy * e) * 1e9

    wavelengths.append(wavelength)
    photonEnergies.append(photonEnergy)

    for panel in (ax, axZoom):
        panel.plot([wavelength, wavelength], [0, photonEnergy],
                   color=colour, linewidth=0.9, alpha=0.75)
```

The subtraction is the right way round because `orbitalEnergy` returns negative values, with the upper level less negative, so the difference comes out positive. The wavelength conversion multiplies by e to get back into joules before applying $\lambda = hc/E$, then by 10^9 for a nanometre axis, which is the same convert-at-the-boundary habit as Task 3.

Drawing into (`ax`, `axZoom`) in the same loop is a small piece of tidiness that saved a lot of trouble. There are two panels, a full view and a zoom, and doing them together means there is only one place where a transition can be added or a colour changed. Two separate loops would have meant keeping two things in step by hand.

I stop at $n = 40$, which is nowhere near infinity but is close enough that the lines have visibly piled up on the series limit and further transitions would add ink without adding information. That limit is worth stating on the figure, since it is hard to read off a crowded axis, so it is computed directly and collected into a text box:

```
# The series limit is the shortest wavelength, reached as n tends to infinity
limitEnergy = -orbitalEnergy(Z, A, nFinal)
limitWavelength = (h * c) / (limitEnergy * e) * 1e9
limitRows.append(f'{name:9s} n->{nFinal}    {limitWavelength:7.1f} nm')
```

Negating the final level energy is the shortcut for taking n_i to infinity, since the upper level energy tends to zero and the gap becomes the whole binding energy of the lower level. The rows are padded with `f'{name:9s}'` and rendered in a monospace box so the wavelengths line up in a column instead of scattering.

The second panel exists because the Lyman and Balmer lines are crammed into the far left of the full view and cannot be told apart there. It plots the same data over 80 to 700 nm and shades the visible range, which picks out the handful of Balmer lines responsible for the colours hydrogen actually emits.

7 Task 6: Electron diffraction

This one chains three relationships together. An electron accelerated through V has momentum $p = \sqrt{2m_e eV}$, so its de Broglie wavelength is

$$\lambda = \frac{h}{\sqrt{2m_e eV}}, \quad (11)$$

Bragg's condition $2d \sin \frac{\phi}{2} = \lambda$ turns that into a scattering angle, and the geometry of the tube turns the angle into a ring radius on the screen, $x = r \sin 2\phi$. All three steps sit in one function:

```
def ringRadius(V, d):
    wavelength = h / np.sqrt(2 * me * e * V)
    sinHalfPhi = wavelength / (2 * d)
    # Only valid where sinHalfPhi <= 1
    sinHalfPhi = np.clip(sinHalfPhi, 0, 0.999)
    phi = 2 * np.arcsin(sinHalfPhi)
    return r * np.sin(2 * phi)
```

The `np.clip` is a guard rather than physics. Rearranging Bragg gives $\sin \frac{\phi}{2} = \lambda/2d$, and if the wavelength were ever more than twice the spacing that ratio would exceed one, `arcsin` would return `nan`, and the curve would develop gaps accompanied by a runtime warning. Between 1 kV and 5 kV it never actually triggers, but since the entire task is about sweeping the voltage I did not want a function that only behaves inside the range I happened to test.

Writing `ringRadius(V, d)` with the spacing as an argument rather than reading a global is what makes the two graphite spacings, 0.123 nm and 0.213 nm, cost nothing extra. They go in a dictionary and everything downstream loops over it.

Recovering the spacing from the gradient

The task asks for a $\sin \frac{1}{2}\phi$ against $1/\sqrt{V}$ graph as a check, and I used it as a real one. Combining the two relations gives

$$\sin \frac{\phi}{2} = \frac{h}{2d\sqrt{2m_e e}} \cdot \frac{1}{\sqrt{V}}, \quad (12)$$

a straight line through the origin whose gradient contains d . Fitting the line and inverting the expression should hand back the spacing I fed in at the top:

```
gradient = np.polyfit(invSqrtV, sinPhi, 1)[0]
dCalculated = h / (2 * gradient * np.sqrt(2 * me * e)) * 1e9
print(f'{label}: gradient = {gradient:.4e}, calculated d = {dCalculated:.3f} nm')
```

`np.polyfit(..., 1)` does a least squares fit of a straight line and returns gradient and intercept, and `[0]` takes the gradient. The values come back as 0.123 nm and 0.213 nm. That closes the loop: the number I put in comes out the other end having been through the de Broglie relation, Bragg's condition and a line fit on the way, so a typo in any of the three would show up here. It is also exactly what you would do with real data from the tube, which is a pleasing coincidence.

The screen panel

The third panel draws what you would actually see, two rings on a dark screen with a bright centre, and a slider changes the voltage:

```
def update(val):
    V = slider.val * 1000
    for (label, d), col in zip(spacings.items(), colours):
        x = ringRadius(V, d) * 1000
        ringCircles[label].set_radius(x)
    voltageText.set_text(f'V = {slider.val:.1f} kV')
    fig.canvas.draw_idle()
```

The callback does not rebuild anything. The two circle patches were stored in the `ringCircles` dictionary when they were created, so the update just reaches for them by name, changes their radii and asks for a redraw. My first attempt cleared the axes and redrew from scratch on every slider event, which stuttered badly and also lost the legend each time.

8 Task 7: Particle in a box

Two panels again, energy against quantum number and probability density against position, using

$$\psi_n(x) = \sqrt{\frac{2}{L}} \sin\left(\frac{n\pi x}{L}\right), \quad E_n = \frac{n^2 \pi^2 \hbar^2}{2m_e L^2}. \quad (13)$$

I set L to the Bohr radius so the box is roughly atom sized, which puts the ground state at about 134 eV.

The energy panel is a single vectorised expression. `nValues` is an array, so every energy comes out at once:

```
nValues = np.arange(1, 6)
energies = (nValues**2 * np.pi**2 * hbar**2) / (2 * me * L**2 * e)
```

Dividing by e at the end converts joules to electronvolts, again at the boundary rather than halfway through the expression. The points are then annotated in a short loop over `zip(nValues, energies)`, which pairs each quantum number with its own energy so the labels cannot end up attached to the wrong point.

The density panel is where the one line I would defend sits:

```
psi = np.sqrt(2/L) * np.sin(n * np.pi * x / L)
psiSquared = psi**2
```

The $\sqrt{2/L}$ is the normalisation factor. It would have been easy to drop it and plot a plain sine squared, since the shape is identical and nobody would notice from the picture. Including it is what makes the vertical axis mean something: with it in place the area under each curve is exactly one, so the three curves are genuinely comparable rather than three arbitrary scalings sharing an axis.

Extension: checking the uncertainty principle

The extension asks me to show that the particle obeys $\Delta x \Delta p \geq \frac{1}{2} \hbar$. This one is worth doing on paper rather than numerically, because every integral involved is tractable. By symmetry $\langle x \rangle = L/2$, and the second moment gives

$$(\Delta x)^2 = L^2 \left(\frac{1}{12} - \frac{1}{2n^2\pi^2} \right). \quad (14)$$

The standing wave is built from equal and opposite travelling waves so $\langle p \rangle = 0$, while $\langle p^2 \rangle = 2m_e E_n$ gives $\Delta p = n\pi\hbar/L$. Multiplying the two,

$$\Delta x \Delta p = \hbar \sqrt{\frac{n^2\pi^2}{12} - \frac{1}{2}}. \quad (15)$$

The box width cancels completely, so the result holds whatever L I put in the code, which is a reassuring thing to find given that L is a variable at the top of the file. For $n = 1$ this comes to $0.568\hbar$, comfortably above $\hbar/2$, and it only grows with n .

9 Task 8: Quantum cryptography

The task asks for a visual calculator comparing two predictions for the mismatch probability when entangled photons are measured at polariser angles θ and ϕ :

$$P_{\text{classical}} = 1 - \cos^2 \theta \cos^2 \phi - \sin^2 \theta \sin^2 \phi, \quad P_{\text{quantum}} = \sin^2(\phi - \theta). \quad (16)$$

Both go in as functions taking the same two arguments:

```
def classicalMismatch(theta, phi):
    return 1 - np.cos(theta)**2 * np.cos(phi)**2 - np.sin(theta)**2 * np.sin(phi)**2

def quantumMismatch(theta, phi):
    return np.sin(phi - theta)**2
```

Giving them matching signatures is the reason the rest of the program is short. Both can be fed the same angles from the same sliders with no special casing, and both accept either a single angle or a whole array, so the same two functions serve the curves and the numerical readout.

Since the two predictions coincide at 0 and 90 degrees, the case worth marking is the one in between where they are furthest apart, and I annotate it directly:

```

thetaEx = np.radians(-30)
phiEx = np.radians(30)
classicalEx = classicalMismatch(thetaEx, phiEx)
quantumEx = quantumMismatch(thetaEx, phiEx)

ax.scatter([60], [classicalEx], color='cyan', s=80, zorder=5)
ax.scatter([60], [quantumEx], color='magenta', s=80, zorder=5)

```

The values are calculated rather than typed in, so the annotation cannot disagree with the curve it is sitting on. They come out as 3/8 and 6/8, a factor of two apart, which is the number the whole figure exists to show. The `zorder=5` lifts the markers above the lines, which otherwise cover them.

The interactive part gives each angle its own slider and updates a text readout:

```

def update(val):
    theta = np.radians(sliderTheta.val)
    phi = np.radians(sliderPhi.val)
    diff = np.linspace(-np.pi, np.pi, 1000)

    classicalLine.set_ydata(classicalMismatch(theta, theta + diff))
    quantumLine.set_ydata(quantumMismatch(theta, theta + diff))
    ...
    fig.canvas.draw_idle()

```

As in Task 6, the update swaps out only the y data of two line objects that already exist, using `set_ydata`, and leaves the axes, labels and legend untouched. The sliders read in degrees and `np.radians` converts at the point of use, since degrees are what a person wants to type and radians are what NumPy expects.

Moving both sliders together is the demonstration I would point someone at: the quantum curve does not move, because it depends only on the difference, while the classical one changes shape.

10 Task 9: Compton scattering

Three panels, all against photon scattering angle, built from

$$\Delta\lambda = \frac{h}{m_e c} (1 - \cos \theta). \quad (17)$$

The Compton wavelength is computed once outside the loop, since it is a constant and there is no reason to recalculate it for every photon energy:

```

comptonWavelength = h / (me * c)
theta = np.linspace(1e-6, np.pi, 1000)

```

The angle array starting at 10^{-6} rather than 0 is deliberate. At exactly zero the recoil angle formula has zero on the top and zero on the bottom, and one millionth of a radian is far too small to see on the axis while being enough to keep the division well behaved.

Everything else follows as a chain, one initial photon energy at a time:

```

E_J = E_keV * 1000 * e
lambda0 = h * c / E_J

```

```

deltaLambda = comptonWavelength * (1 - np.cos(theta))
lambda1 = lambda0 + deltaLambda

fractionalShift = deltaLambda / lambda0

E1_J = h * c / lambda1
electronKE = E_J - E1_J

```

`lambda0` is a single number and `deltaLambda` is an array of a thousand values, so `lambda0 + deltaLambda` broadcasts the scalar across the array and gives the scattered wavelength at every angle in one addition. The electron kinetic energy is then whatever the photon lost.

The recoil speed has to be relativistic. At these energies the electron picks up a decent fraction of its 511 keV rest energy, and the classical $v = \sqrt{2E_k/m_e}$ will cheerfully return speeds above c :

```

gamma = 1 + electronKE / (me * c**2)
recoilSpeed = c * np.sqrt(1 - 1/gamma**2)

```

Going through the Lorentz factor means the result is bounded by construction, since $\gamma \geq 1$ always, so the expression cannot produce something faster than light no matter what energy I put in. Plotting `recoilSpeed / c` rather than the raw speed makes that bound visible as a line the curves approach but never cross.

For the recoil angle I kept the exact formula rather than any small angle version, since the whole point of the panel is the behaviour at large angles where an approximation would fall over:

```

numerator = np.sin(theta)
denominator = 1 + (h / (me * c * lambda0)) * (1 - np.cos(theta)) - np.cos(theta)
phi = np.arctan(numerator / denominator)

```

Splitting it into `numerator` and `denominator` was purely so I could read it. Written as one line it is a wall of brackets and I mistyped it twice before giving the two halves names.

11 Task 10: Hydrogenic orbitals, two dimensional slices

The functions this task draws all come from `hydrogenic`, the module I wrote for Task 5 and extended here:

```

from hydrogenic import (probabilityDensity, orbitalEnergy,
                        orbitalLetter, atomRadius)

```

How the wavefunction is evaluated

`probabilityDensity` takes spherical coordinates along with Z , A , n , ℓ and m and gives back $|\psi|^2$. Everything below it is about feeding it the right coordinates and drawing the answer without misleading anyone, but it is worth setting out what the module does on the way, since this task is the one that uses all of it.

The wavefunction separates into a radial and an angular factor, $\psi_{n\ell m} = R_{n\ell}(r) Y_{\ell}^m(\theta, \phi)$, and the two are computed independently before being multiplied together in `hydrogenicWavefunction`.

The radial part is

$$R_{n\ell}(r) = \sqrt{\frac{(n-\ell-1)!}{2n(n+\ell)!}} \left(\frac{2}{an}\right)^{3/2} x^\ell e^{-x/2} L_{n-\ell-1}^{2\ell+1}(x), \quad x = \frac{2r}{an}, \quad (18)$$

and the awkward object in it is the associated Laguerre polynomial. Rather than reach for a library version I evaluated it directly as its finite sum,

$$L_{n-\ell-1}^{2\ell+1}(x) = \sum_{k=0}^{n-\ell-1} \frac{(\ell+n)! (-x)^k}{(2\ell+1+k)! (n-\ell-1-k)! k!}, \quad (19)$$

which is what `laguerreSum` implements:

```
def laguerreSum(x, n, l):
    total = np.zeros_like(x, dtype=float)
    for k in range(n - l):
        coeff = (factorial(l + n) * (-x)**k) / (
            factorial(2*l + 1 + k) * factorial(n - l - 1 - k) * factorial(k)
        )
        total = total + coeff
    return total
```

The sum terminates after $n - \ell$ terms, so the loop is short and its length is fixed by the quantum numbers rather than by any tolerance I have to choose. `x` arrives as the whole coordinate array and each term is added to the running total across all of it at once, so the loop runs over polynomial order and never over grid points. For the orbitals plotted here that is Every orbital in the list plotted here has $\ell = n - 1$, so $n - \ell$ is one and the sum reduces to a single term in each case; the loop only does real work for states with $\ell < n - 1$. array to evaluate the polynomial everywhere.

The angular part is built from the spherical harmonics

$$Y_\ell^m(\theta, \phi) = (-1)^m \sqrt{\frac{2\ell+1}{4\pi} \cdot \frac{(\ell-|m|)!}{(\ell+|m|)!}} P_\ell^{|m|}(\cos \theta) e^{im\phi}, \quad (20)$$

with the associated Legendre function taken from `scipy.special.lpmv` and everything else written out. That one import is the only piece of special-function machinery I did not implement myself.

Left as it stands, Y_ℓ^m is complex for $m \neq 0$, and a complex density is not something `contourf` can draw. `angularPart` therefore combines the $\pm m$ harmonics into real-valued pairs:

```
def angularPart(theta, phi, l, m):
    if m == 0:
        return sphericalHarmonic(theta, phi, l, 0)
    elif m < 0:
        return (sphericalHarmonic(theta, phi, l, -m)
                - sphericalHarmonic(theta, phi, l, m))
    else:
        return (sphericalHarmonic(theta, phi, l, m)
                + sphericalHarmonic(theta, phi, l, -m))
```

Since Y_ℓ^m and Y_ℓ^{-m} are degenerate, any combination of the two is an equally valid state, and the sum and difference are the ones that come out real. These are the orbitals drawn in chemistry textbooks: p_x and p_y rather than $m = \pm 1$. It is a change of basis and not a change

of physics, which is worth saying plainly, because the pictures in this section are of the real combinations rather than of Y_ℓ^m itself.

`probabilityDensity` then applies the Born interpretation as `np.real(psi * np.conj(psi))`. The `np.real` is not redundant even after the combinations above: multiplying by the conjugate leaves a zero imaginary part in exact arithmetic but a residue around 10^{-40} in floating point, and stripping it is what stops the array staying complex-typed all the way into the plotting call.

One guard sits at the top of `hydrogenicWavefunction`:

```
if not (0 <= l <= n - 1 and abs(m) <= l):
    raise ValueError(f'Invalid quantum numbers: n={n}, l={l}, m={m}')
```

An invalid combination does not fail cleanly on its own. A negative factorial argument returns `inf` rather than raising, and the result is a plot of nothing with no explanation. Failing at the point where the impossible state was requested was worth the three lines, and it is what the interactive controls in the app rely on to constrain ℓ and m to the ranges that exist.

Sizing the axes to the orbital

This was the real problem in the task and I underestimated it at first. Orbitals differ enormously in size. A $1s$ is essentially contained within a Bohr radius while a $4f$ reaches more than twenty times further, so a fixed axis range that shows one properly renders the other as a dot or cuts it in half. Scaling as n^2 gets close, but not close enough, because l and m move the density around as well.

Rather than guess, I measure it. The routine lays down a coarse probe grid, finds where the normalised density is above the display threshold, and asks how far out that region reaches:

```
def findExtent(Z, A, n, l, m):
    # Orbital size grows roughly with n squared but also depends on l and m,
    # so a coarse probe grid is used to size each panel to its own orbital
    trial = 2.5 * n**2 * atomRadius(Z, A)

    for _ in range(14):
        probe = np.linspace(-trial, trial, 40)
        X, ZZ = np.meshgrid(probe, probe, indexing='ij')
        r = np.sqrt(X**2 + ZZ**2)
        r[r == 0] = 1e-20
        theta = np.arccos(np.clip(ZZ / r, -1, 1))
        phi = np.zeros_like(r)

        density = probabilityDensity(r, theta, phi, Z, A, n, l, m)
        density = density / density.max()
        keep = density > threshold

        if keep.sum() == 0:
            trial = trial * 1.4
            continue

        reach = max(abs(X[keep]).max(), abs(ZZ[keep]).max())
        if reach < 0.72 * trial:
            return reach * 1.35
        trial = trial * 1.35

    return trial
```


There are two ways out of the loop. If the probe finds nothing above threshold then the window is too small to contain anything visible, so it grows by forty per cent and tries again. If it does find something, `reach` is how far that something extends, and the test `reach < 0.72 * trial` asks whether the orbital fits comfortably inside the current box. If it does, the window is settled and returned with a thirty five per cent margin so the outer contour is not pressed against the axis. If it does not, the orbital is probably still being clipped, so the box grows and the search goes round again. The fourteen iteration cap means it cannot hang if something unexpected comes in.

The probe grid is only 40×40 because it is only answering a yes or no question about size. The expensive high resolution evaluation happens once afterwards, on a window already known to be right. Doing the search at full resolution would have multiplied the cost of every panel for no gain.

Two small guards inside are worth pointing out. `r[r == 0] = 1e-20` avoids a division by zero at the nucleus, which any grid containing the origin will hit. `np.clip(ZZ / r, -1, 1)` protects the `arccos`: that ratio can never leave $[-1, 1]$ mathematically, but in floating point it can come out as `1.0000000000000002`, and `arccos` of that is `nan`. Both of these produced mysterious blank panels before I worked out what was happening.

Getting the azimuthal angle right in the slice

The slice is taken through the $y = 0$ plane, so the panel axes are x and z and the Cartesian grid has to be converted into the spherical coordinates `probabilityDensity` wants. The line that matters is this one:

```
# In the y = 0 plane phi is 0 where x is positive and pi where x is negative
phi = np.where(X >= 0, 0.0, np.pi)
```

My first version set $\phi = 0$ everywhere, which felt reasonable and is wrong. In the $y = 0$ plane the azimuthal angle is 0 on the half where $x > 0$ and π on the half where $x < 0$. Since the spherical harmonics carry a factor $e^{im\phi}$, any orbital with $m \neq 0$ is genuinely different across those two halves. Setting it to zero throughout produced a plot with a mirror symmetry that does not exist, and it looked entirely plausible, which is what made it worth catching.

Each panel is normalised to its own maximum and drawn with sixty filled contour levels, enough that the density reads as continuous while keeping the nodal structure crisp:

```
image = ax.contourf(X * 1e10, ZZ * 1e10, density,
                    levels=60, cmap='jet', vmin=0, vmax=1)
```

The `* 1e10` converts metres to angstroms for the axes, while the density itself was computed in metres, which is the same boundary conversion rule as everywhere else.

Six orbitals are laid out on a grid built from `columns = 3` and a row count worked out with `np.ceil`, so changing the list of orbitals at the top does not require touching the layout code. `axes.flatten()` turns the two dimensional array of axes into a flat one so a single index walks through them, and any leftover panels are switched off:

```
for spare in range(len(orbitals), len(axes)):
    axes[spare].axis('off')
```

The titles print the energy from `orbitalEnergy` alongside the quantum numbers, which is how the $m = 0$ and $m = 2$ panels can be seen to look completely different while carrying identical energies.

12 Task 10: Hydrogenic orbitals, three dimensional

The 3D version shares `findExtent` with the 2D one, adapted to three axes, and then has two problems of its own.

Building the volume out of separated planes

The obvious approach is an isosurface, but an isosurface shows a single density value and is opaque, so it hides everything inside itself. For a 3d orbital most of what I want to show is the nodal structure and the bright core, so that is precisely the wrong thing to throw away.

Instead I build the volume as a stack of flat z planes with gaps between them, each drawn as a low alpha scatter, so the eye can see through the outer regions:

```
# The volume is built as a stack of flat z planes rather than a solid grid.  
# Leaving gaps between the planes lets you see through the outer surface  
# to the higher density core, which a filled volume would completely hide.  
sliceCount = 26  
planeResolution = 130
```

Those two numbers are a trade I settled by trying several. Twenty six planes is enough that the stack reads as a solid shape from most angles but sparse enough to see between, and 130×130 in the plane keeps each slice smooth without the whole thing taking minutes.

One normalisation for the whole volume

This is the correctness point of the 3D plot and it is invisible if you get it wrong. If each slice were normalised against its own maximum, the way the 2D panels are, then a slice way out in the tail holding almost no probability would be rescaled until its feeble peak was drawn in the same bright red as the true centre. The colour bar would be meaningless and the picture would badly misrepresent where the electron actually is.

So the peak is found once, over the whole volume, and every slice is divided by that same number:

```
def peakDensity(Z, A, n, l, m, extent):  
    # Every slice must be scaled against the same peak value, otherwise each  
    # one would be normalised separately and the colours would not compare  
    probe = np.linspace(-extent, extent, 44)  
    X, Y, ZZ = np.meshgrid(probe, probe, probe, indexing='ij')  
    r = np.sqrt(X**2 + Y**2 + ZZ**2)  
    r[r == 0] = 1e-20  
    theta = np.arccos(np.clip(ZZ / r, -1, 1))  
    phi = np.arctan2(Y, X)  
    return probabilityDensity(r, theta, phi, Z, A, n, l, m).max()
```

Note `np.arctan2(Y, X)` rather than `np.arctan(Y / X)`. The two argument version knows which quadrant the point is in and returns an angle over the full 2π , whereas the single argument version folds the left and right halves of space on top of each other. It is the same trap as the ϕ line in the 2D slice, in a different disguise.

The scatter call then locks the colour scale as well, so a given colour means the same density in every plane:

```
scatter = ax.scatter(  
    X[keep] * 1e10,  
    Y[keep] * 1e10,
```

```

np.full(keep.sum(), zValue * 1e10),
c=density[keep], cmap='jet', s=4, alpha=0.16,
vmin=threshold, vmax=1, edgecolors='none', depthshade=False
)

```

`X[keep]` pulls out only the points above threshold, which is usually a small fraction of the plane, so the number of markers drawn stays manageable. `np.full(keep.sum(), zValue * 1e10)` builds the z coordinate, since every point in a given plane shares the same height and `scatter` still wants an array as long as the other two. `depthshade=False` stops Matplotlib fading distant markers, which would otherwise be a second, unwanted brightness scale fighting the colour map.

The threshold of 0.15 is doing real work too. The wavefunctions never actually reach zero, so drawing everything gives a solid block. Cutting at fifteen per cent of the peak shows the region the electron occupies in practice. That is a display choice rather than a physical statement, which is exactly why it is a named variable at the top of the file rather than a number buried in a plotting call.

13 Extension: making all ten interactive

Every task ended up with controls attached, which was much less work than it sounds because of the habit described at the start. The physics in each task already lived in a function that took parameters and returned numbers, so making something interactive only ever meant calling the same function with a different argument and redrawing. Tasks 6 and 8 were done inside the original scripts with Matplotlib's `Slider` widget and an update callback, and the rest followed the same pattern.

The parameters I exposed are the ones that change the physics rather than the appearance:

- Task 1: number of steps, step size and number of overlaid walks.
- Task 2: temperature, mass ratio, particle count, K_n and the coefficient of restitution. Dropping C below one is the most instructive control in the set, since the big particle visibly settles.
- Task 3: temperature for the Planck curve and Einstein temperature for the heat capacity.
- Task 4: work function and choice of metals, with the threshold sliding along while the gradient stays put.
- Task 5: atomic number Z , the highest transition followed, and which series are shown.
- Task 6: accelerating voltage from 1 kV to 5 kV, with the screen rings resizing live.
- Task 7: quantum number and box width.
- Task 8: both polariser angles, with a live numerical readout.
- Task 9: incident photon energy.
- Task 10: n , l , m , Z and the display threshold, with the controls constrained so that l only runs from 0 to $n - 1$ and m from $-l$ to $+l$. That constraint matters, because otherwise it is possible to ask the interface for a state that does not exist and get an unhelpful crash.

Putting them in one place

Ten scripts that each open their own window is fine while developing but means ten programs and a Python installation for anybody who wants to look. So I combined them into a single web

application built with Streamlit and deployed on Streamlit Community Cloud, which runs in a browser with nothing to install.

Most of the migration was mechanical. Each `Slider` became a `st.slider` in the sidebar, and every `update` callback disappeared entirely, because Streamlit re-runs the script from the top whenever a control moves rather than patching an existing figure. Each task became a module exposing a function that takes the control values and hands back a Matplotlib figure, which the app passes to `st.pyplot`. The physics functions themselves crossed over unchanged, which is the clearest evidence I have that separating them from the drawing code was the right call early on.

Two things needed real work. The Brownian animation could not use `FuncAnimation`, which wants an interactive backend, so the trajectory is computed for a chosen number of frames and then either shown as a completed path or encoded for playback. And the slower computations, particularly the 3D orbital volume, are wrapped in Streamlit's caching decorator so that touching an unrelated control does not trigger a recalculation that has already been done once.

The result is one page with a task selector, the controls for whichever task is showing, and a figure that updates as they move.

The app is live at <https://bpho-quantum-2026-bryan-okafor.streamlit.app/>